

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:
Губанов Егор
Группа: БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026

Задача

Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и мой)

С учётом имеющихся данных по результатам Д31 и Д32 реализуйте:

Модели, представления, контроллеры

В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Необходимо сделать отчёт по шаблону

Пример бойлерплейта: `express-typeorm-boilerplate` Пример реализации REST

API: `express-microservices-example`

Ход работы

1. Используемые технологии

Для реализации выбран следующий технологический стек:

Компонент	Версия / решение
Node.js	Среда выполнения (локальная установка)
TypeScript	^5.6
Express	^4.21
TypeORM	^0.3.20, драйвер pg ^8.13
Аутентификация	jsonwebtoken ^9.0.2, хэширование паролей – bcryptjs
Запуск в разработке	ts-node-dev (npm run dev)
Сборка	tsc → npm run build; продакшен-старт: node dist/index.js
База данных	PostgreSQL (через Docker Compose)
Документация	Swagger UI из Д32, порт 8001

Выбор Express + TypeORM обусловлен простотой интеграции с TypeScript и соответствием задачам учебного проекта. `synchronize: true` в конфигурации TypeORM позволяет автоматически поднимать схему БД при старте без написания миграций, что удобно для лабораторного стенда.

2. Архитектура приложения

Проект организован по принципу разделения ответственности (separation of concerns). Структура файлов:

Файл / директория	Назначение
src/index.ts	Точка входа: dotenv, express.json(), CORS, роутинг, errorHandler, AppDataSource, сид типов, listen.
src/routes.ts	Декларация всех маршрутов с привязкой к контроллерам.
src/controllers/*	Обработка HTTP: разбор body/query/params, вызов TypeORM, формирование ответа через сериализаторы.
src/serializers.ts	Приведение сущностей к полям в стиле OpenAPI (snake case).
src/middleware/auth.ts	Извлечение JWT, загрузка пользователя, requireAuth.
src/http/errors.ts	Классы HTTP-ошибок (401/403/404/409/422/500).
src/middleware/errorHandler.ts	Глобальный обработчик: единый формат {code, message}.

src/data-source.ts	Конфигурация PostgreSQL; synchronize: true для учебного стенда.
--------------------	---

Все маршруты подключаются в src/index.ts через app.use("/api/v1", buildRoutes()). Для части путей в роутере продублированы варианты с завершающим "/" и без – во избежание ошибок 404 из-за различий в клиентских запросах.

3. Модель данных

В AppDataSource зарегистрированы следующие сущности:

Сущность	Описание
User	Пользователь: роль TENANT OWNER ADMIN, email, хэш пароля, профиль.
PropertyType	Тип недвижимости: название, флаг публикации. При старте заполняется сидом (Квартира, Дом, Комната).
Property	Объект недвижимости: владелец, тип, адрес, цена, флаг публикации. Связан с Photo, RentalCondition, Deal, Message.
Photo	Фотография объекта (URL).
RentalCondition	Текстовое условие аренды с порядком сортировки.
Deal	Сделка: объект, арендатор, статус PENDING / ACTIVE / COMPLETED, даты, сумма.
Message	Сообщение между пользователями в контексте объекта.
RefreshToken	Refresh-сессия пользователя (хранится в БД).
PasswordResetToken	Одноразовый токен сброса пароля.

4. Реализованные эндпоинты

Базовый URL для всех запросов: http://localhost:3000/api/v1

Метод	Путь	Назначение	Авторизация
POST	/api/v1/auth/register	Регистрация	нет
POST	/api/v1/auth/login	Вход, получение токенов	нет
POST	/api/v1/auth/refresh	Обновление access-токена	нет
POST	/api/v1/auth/logout	Выход, удаление refresh-сессии	Bearer
POST	/api/v1/auth/password/request-reset	Запрос токена сброса пароля	нет
POST	/api/v1/auth/password/reset	Сброс пароля по токenu	нет
GET	/api/v1/me	Текущий пользователь	Bearer
GET	/api/v1/me/renting	Сделки, где я – арендатор	Bearer
GET	/api/v1/me/owning	Мои объекты и активные сделки	Bearer
GET	/api/v1/me/owning/deals	Все сделки по моим объектам	Bearer
GET	/api/v1/property-types	Справочник типов недвижимости	нет
GET	/api/v1/properties	Список/поиск (фильтры в query)	нет
GET	/api/v1/properties/:id	Карточка объекта	нет
POST	/api/v1/properties	Создать объявление	Bearer

PUT	/api/v1/properties/:id	Обновить объявление	Bearer
DELETE	/api/v1/properties/:id	Удалить объявление	Bearer
POST	/api/v1/properties/:id/photos	Добавить фото	Bearer
DELETE	/api/v1/photos/:id	Удалить фото	Bearer
POST	/api/v1/deals	Создать заявку на аренду	Bearer
GET	/api/v1/deals/:id	Получить сделку	Bearer
PATCH	/api/v1/deals/:id	Сменить статус сделки	Bearer
GET	/api/v1/messages	Список сообщений	Bearer
POST	/api/v1/messages	Отправить сообщение	Bearer
PATCH	/api/v1/messages/:id/read	Пометить прочитанным	Bearer
GET	/api/v1/history	Агрегат: сообщения и сделки	Bearer

5. Аутентификация и авторизация

Схема аутентификации построена на двух типах токенов:

- Access-токен (JWT) – короткоживущий, передаётся в заголовке Authorization: Bearer <token>. Используется для авторизации всех защищённых запросов.
- Refresh-токен – долгоживущая строка, хранится в таблице RefreshToken в БД. Используется для получения новой пары токенов через POST /auth/refresh без повторного ввода пароля.
- Логика авторизации:
- Middleware requireAuth извлекает JWT из заголовка, верифицирует подпись и загружает пользователя из БД.
- Проверка владельца ресурса выполняется на уровне контроллера: сравнивается id пользователя из токена с owner_id объекта/сделки.
- Ролевые ограничения (например, создание сделки только для TENANT, смена статуса только для OWNER) реализованы в контроллерах.

Сброс пароля: клиент отправляет POST /auth/password/request-reset с email. В учебной реализации токен сброса выводится в консоль сервера. Затем POST /auth/password/reset с полями reset_token и new_password завершает процедуру.

6. Система ошибок

Все ошибки возвращаются в едином формате, соответствующем контракту OpenAPI:

```
{
  "code": 404,
  "message": "Объект недвижимости не найден"
}
```

Таблица кодов ошибок:

HTTP-код	Условие возникновения
200	Успешный запрос
401	Нет токена, неверный логин или пароль
403	Недостаточно прав: не та роль или не владелец ресурса
404	Объект, сделка, сообщение или фото не найдены

409	Конфликт: email занят, недопустимый переход статуса сделки
422	Невалидное тело запроса или параметры
500	Внутренняя ошибка сервера

Классы ошибок (NotFoundError, ForbiddenError и др.) определены в src/http/errors.ts и перехватываются глобальным errorHandler, который формирует единообразный JSON-ответ.

7. Тестирование

Проверка выполнялась запросами к локальному API через Postman, Swagger Try it out и curl. Проверенные сценарии:

- Регистрация пользователей с ролями OWNER и TENANT.
- Вход, получение access- и refresh-токена; обновление токена через /auth/refresh.
- Создание объекта недвижимости с фото и условиями аренды.
- Публичный листинг объектов с фильтрами (город, тип, диапазон цены, пагинация).
- Просмотр карточки объекта.
- Создание заявки на аренду арендатором, смена статуса владельцем.
- Переписка по объекту: отправка и прочтение сообщений.
- Просмотр профиля (/me), истории сделок (/me/renting, /me/owning/deals) и сводной истории (/history).

В репозитории в каталоге homeworks/hw3 хранится коллекция Postman с E2E-проверками сквозных сценариев.

Проверенные коды ответов: 200 при успехе, 401 без/с неверным токеном, 403 при нарушении прав, 404 при отсутствии ресурса, 409 при конфликте (недопустимый переход статуса сделки, занятый email), 422 при невалидных данных.

Вывод

В ходе лабораторной работы реализован полноценный REST API на Express + TypeORM + PostgreSQL, соответствующий OpenAPI-контракту курса с базовым префиксом /api/v1. Реализованы все основные группы эндпоинтов: аутентификация с access- и refresh-токенами, сброс пароля, управление объектами недвижимости и фото, сделки со сменой статусов, переписка, профиль и история. Разграничение прав по ролям (TENANT, OWNER, ADMIN) обеспечивается на уровне middleware и контроллеров. Настроен запуск через Docker Compose и переменные окружения .env. Применение synchronize: true обосновано учебным характером работы; в продакшен-среде необходимо использовать версионированные миграции. Полученный стенд удобен для проверки через Postman или Swagger UI с локального хоста.